

ASSESSMENT-1

Selenium Installation and Troubleshooting Assignment

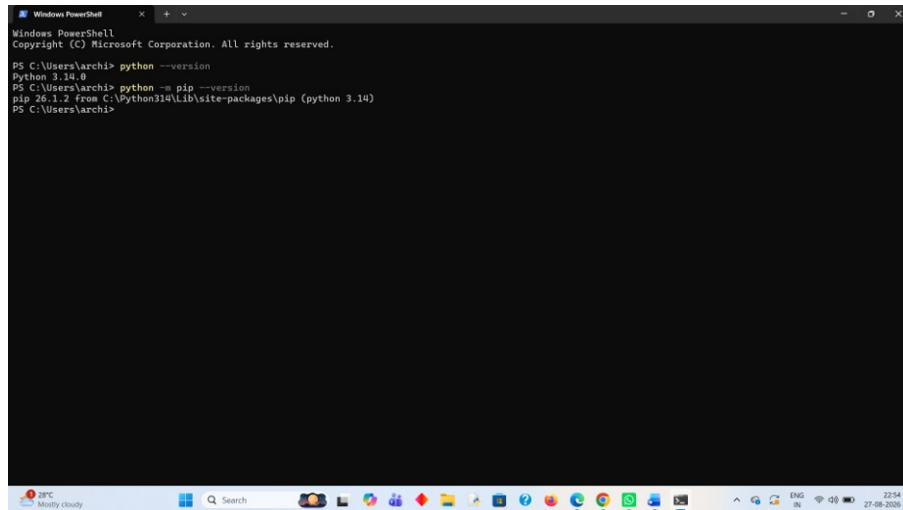
Date:27.06.2026

**Name-Senjuti Khan ,Sec-A,Roll-48 Enrollment
no-12023002001154**

Part A – Installation and Verification Tasks

Q1. Record the operating system, Python version, pip version, browser name, and browser version.

Answer:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\archi> python --version
Python 3.14.0
PS C:\Users\archi> python -m pip --version
pip 26.1.2 from C:\Python314\Lib\site-packages\pip (python 3.14)
PS C:\Users\archi>
```

Figure 1: Python installation

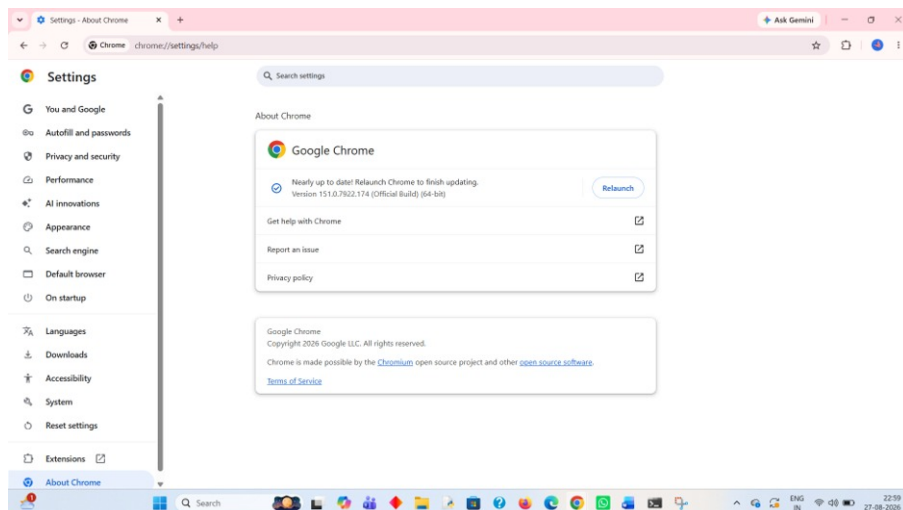


Figure 2: Browser version

Q2. Create and Activate Virtual Environment

Answer:

```

MINGW64~/Users/arch/selenium_assignment
sch@archita MINGW64 ~
$ python --version
Python 3.14.0

sch@archita MINGW64 ~
$ python -m pip --version
pip 26.1.2 from C:\Python14\lib\site-packages\pip (python 3.14)

sch@archita MINGW64 ~
$ mkdir selenium_assignment

sch@archita MINGW64 ~
$ cd selenium_assignment

sch@archita MINGW64 ~/selenium_assignment
$ python -m venv venv
Creating virtual environment in venv/
$ source venv/bin/activate
bash: venv/bin/activate: No such file or directory

sch@archita MINGW64 ~/selenium_assignment
$ python1 -m venv venv
Python was not found: run without arguments to install from the Microsoft Store,
or disable this shortcut from Settings > Apps > Advanced app settings > App exe
cution aliases.

sch@archita MINGW64 ~/selenium_assignment
$ AC

sch@archita MINGW64 ~/selenium_assignment
$ source venv/Scripts/activate
(venv)

sch@archita MINGW64 ~/selenium_assignment
$ pip install selenium
Collecting selenium
  Downloading selenium-4.47.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi<2026.2.2 (from selenium)
  Downloading certifi-2026.2.2-py3-none-any.whl.metadata (2.5 kB)
Collecting trio<0.30.0, >0.25.0 (from selenium)
  Downloading trio-0.29.0-py3-none-any.whl.metadata (8.5 kB)
Collecting trio-websocket<0.12.2, >0.11.0 (from selenium)
  Downloading trio_websocket-0.12.2-py3-none-any.whl.metadata (5.1 kB)
Collecting typing_extensions<4.10.0, >4.9.0 (from selenium)
  Downloading typing_extensions-4.10.0-py3-none-any.whl.metadata (2.3 kB)
Collecting urllib3<2.0.0, >1.26.0 (from selenium)
  Downloading urllib3-2.0.7-py3-none-any.whl.metadata (6.9 kB)
Collecting websocket-client<2.0.0, >1.8.0 (from selenium)
  Downloading websocket_client-1.8.0-py3-none-any.whl.metadata (8.3 kB)
Collecting attrs<25.1.0, >25.0.0 (from selenium)
  Downloading attrs-25.1.0-py3-none-any.whl.metadata (8.8 kB)
Collecting sortedcontainers (from selenium)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Collecting idna (from selenium)
  Downloading idna-3.10-py3-none-any.whl.metadata (9.2 kB)
Collecting outcome (from selenium)
  Downloading outcome-1.3.0-py2.py3-none-any.whl.metadata (2.6 kB)

```

Figure 3: : creating virtual enviornment

Q3.Install Selenium

Answer:

Q4. Script Creation and Initial Run

Answer:

Q5. Explain Driver Discovery Run Twice

Answer:

Q6. 6. Reproducibility Environment Expor

Answer:

“‘latex

Part B – Technical Investigation Questions

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

Answer:

Selenium is an open-source tool used to automate web browsers. It is mainly used for testing web applications by performing actions such as opening a website, clicking buttons, entering data, and checking results.

Suitable use cases:

1. Testing a login page by entering a username and password.
2. Testing an e-commerce website by searching for a product and adding it to the cart.

Unsuitable use cases:

1. Testing desktop applications such as Microsoft Word.

2. Directly testing hardware devices.

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

Answer:

The three main Selenium tools have different purposes:

- **Selenium WebDriver:** It is used to automate browsers through programming languages such as Python, Java, and C#. It gives more control over the test.
- **Selenium IDE:** It is a browser extension that can record actions performed on a website and replay them. It is useful for simple tests and beginners.
- **Selenium Grid:** It is used to run tests on different browsers, operating systems, or computers at the same time.

In simple terms, WebDriver is mainly for coding automation, IDE is for record-and-playback automation, and Grid is for running tests in parallel.

Q3. Explain the roles of the test script, Selenium client library, browser driver, browser, and WebDriver protocol during one command such as `driver.get()`.

Answer:

Suppose the Python program contains:

```
driver.get("https://example.com")
```

The command goes through the following steps:

1. **Test script:** The Python program tells Selenium to open the website.
2. **Selenium client library:** The Selenium Python library converts the Python command into a WebDriver request.
3. **WebDriver protocol:** The request is sent using the WebDriver standard.
4. **Browser driver:** The browser driver receives the request and communicates with the browser.
5. **Browser:** The browser finally opens the requested URL.

So, the test script gives the command, Selenium sends the request, the driver communicates with the browser, and the browser performs the action.

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

Answer:

A virtual environment creates a separate environment for a Python project. It allows us to install the required packages and their versions without affecting other projects.

Installing packages globally can create problems when two projects need different versions of the same package. Updating a package for one project might then cause another project to stop working.

For example, if one project needs an older Selenium version and another needs a newer version, separate virtual environments can be used for both projects.

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

Answer:

Selenium Manager is a tool included with recent Selenium versions that helps manage browser drivers automatically.

Earlier, we usually had to download the correct browser driver manually and set its path. With Selenium Manager, Selenium can detect the browser and manage the required driver automatically in many normal situations.

This makes setup much easier and reduces common driver-path and version problems. However, problems can still occur if the network uses a proxy or firewall, or if the browser is installed in an unusual way.

Q6. Differentiate `driver.find_element()` and `driver.find_elements()`. What is returned when no element matches?

Answer:

`driver.find_element()` is used when we want to find one element. If no matching element is found, Selenium raises a `NoSuchElementException`.

`driver.find_elements()` is used when we want to find multiple elements. It returns a list of matching elements. If nothing is found, it returns an empty list.

For example:

```
element = driver.find_element(By.ID, "username")
```

```
elements = driver.find_elements(By.TAG_NAME, "input")
```

Q7. Compare ID, name, class name, CSS selector, and XPath locators. Which locator strategy would you prefer for stable tests, and why?

Answer:

Different locators are used to identify elements on a web page:

- **ID:** Uses the element's unique `id`. It is usually a good and stable choice.
- **Name:** Uses the element's `name` attribute.
- **Class name:** Uses the element's CSS class. It may not be stable if the class names are generated dynamically.
- **CSS selector:** Uses CSS syntax to identify an element. It is flexible and usually easy to maintain.

- **XPath:** Can locate elements using their structure, attributes, text, or relationships with other elements. It is powerful but complicated XPath expressions can be difficult to maintain.

For stable tests, I would prefer a unique ID if one is available. Otherwise, I would use a simple and stable CSS selector or another meaningful attribute.

Q8. Differentiate implicit wait, explicit wait, and `time.sleep()`. Give one appropriate use case for each.

Answer:

- **Implicit wait:** It tells Selenium to wait for a certain amount of time when looking for elements. It applies to element searches throughout the driver session.

Use case: It can be useful when elements on a website usually take a short time to appear.

- **Explicit wait:** It waits for a particular condition, such as an element becoming visible or clickable.

Use case: It is useful when a button becomes clickable only after some data has loaded.

- **`time.sleep()`:** It simply pauses the program for a fixed amount of time.

Use case: It can be useful temporarily while debugging, but explicit waits are normally better for actual tests.

Q9. Why should `driver.quit()` normally be placed in a finally block or teardown method? How is it different from `driver.close()`?

Answer:

`driver.quit()` is normally placed in a **finally** block so that it runs even if an error occurs during the test. This makes sure that the browser session is properly closed.

`driver.quit()` closes the complete browser session and all windows opened by that WebDriver instance.

On the other hand, `driver.close()` closes only the currently active browser window or tab.

For example:

```
try:
    driver.get("https://example.com")
finally:
    driver.quit()
```

Q10. What causes `StaleElementReferenceException`? Describe two robust ways to handle it without hiding a genuine application defect.

Answer:

A `StaleElementReferenceException` happens when Selenium is trying to use an element that is no longer connected to the current page. This can happen if the page is refreshed or JavaScript replaces the element.

Two ways to handle it are:

1. **Find the element again:** Instead of using the old WebElement, locate the element again after the page changes.
2. **Use an explicit wait:** Wait for the required element or condition before interacting with it. This helps avoid acting before the page has finished updating.

The exception should not simply be ignored because it could indicate a real timing or application problem.

Q11. What is headless browser execution? Identify two benefits and two limitations.

Answer:

Headless execution means running a browser without showing its normal graphical window on the screen.

Benefits:

1. It can use fewer system resources because there is no visible browser window.
2. It is useful on servers or CI/CD systems where a normal graphical browser may not be available.

Limitations:

1. Debugging can be more difficult because we cannot see the browser directly.
2. A website may behave slightly differently in headless mode because of differences in window size, rendering, or browser settings.

Q12. Explain how browser and driver version mismatch can affect a Selenium session. Is this still relevant when Selenium Manager is used? Justify.

Answer:

The browser driver needs to be compatible with the browser it controls. If the driver and browser versions are incompatible, the Selenium session may fail to start or some commands may not work properly.

Selenium Manager has reduced this problem because it can automatically manage the required driver in recent Selenium versions.

However, the problem can still occur in some situations, such as when the browser is installed in a custom location, the network blocks automatic downloads, or an unsupported browser version is being used.

Therefore, Selenium Manager makes driver setup easier, but it does not completely remove all compatibility problems.

Q13. Why can a script work on one computer but fail on another? Discuss at least four environment-related reasons.

Answer:

A Selenium script may work on one computer and fail on another because the two computers may have different environments.

Some common reasons are:

1. Different Python or Selenium versions.
2. Different browser versions.
3. Different operating systems or system settings.
4. Missing or incompatible browser drivers.
5. Different packages or virtual environments.

6. Proxy, firewall, antivirus, or network restrictions.
7. Different browser window sizes or browser settings.

So, when reproducing a Selenium problem, we should compare the Python version, Selenium version, browser version, operating system, and network settings.

Q14. What information should be collected before reporting a Selenium defect so another person can reproduce it?

Answer:

Before reporting a Selenium defect, I would collect the following information:

1. Steps that reproduce the problem.
2. The relevant Selenium/Python code.
3. The website URL, if it is safe and allowed to share.
4. Python, Selenium, browser, and operating system versions.
5. The complete error message and stack trace.
6. Browser driver details, if applicable.
7. Screenshots, logs, or videos of the failure.
8. The expected result and the actual result.
9. Any special settings, such as proxy usage or headless mode.

This information makes it much easier for another person to reproduce and understand the problem.

Q15. Identify three ethical or security precautions required when automating a website.

Answer:

Three important precautions are:

1. **Take permission:** We should automate only websites where we have permission to perform testing.
2. **Protect sensitive information:** Passwords, personal information, tokens, and other confidential data should not be exposed in the code, screenshots, or logs.
3. **Avoid overloading the website:** The automation should use reasonable delays and request rates so that it does not put unnecessary load on the website.

Part C – Installation Challenges to Solve

C1 – pip is not recognized

Problem: The terminal reports that ‘pip’ is not recognized or the command cannot be found.

Solution:

First, check whether Python is installed:

```
python --version
```

If Python is installed, try running pip through Python:

```
python -m pip --version
```

If this works, install Selenium using:

```
python -m pip install selenium
```

This is often better than using only the `pip` command because it makes sure that pip belongs to the Python installation being used.

If Python itself is not recognized, Python may not be installed correctly or may not be added to the system PATH.

C2 – Package installed, import fails

Problem: pip reports a successful Selenium installation, but the IDE shows ModuleNotFoundError: No module named ‘selenium’.

Solution:

This usually happens when Selenium is installed in one Python environment but the IDE is using another one.

First check which Python is being used:

```
python --version
python -c "import sys; print(sys.executable)"
```

Then install Selenium using that same Python:

```
python -m pip install selenium
```

In VS Code or PyCharm, make sure the selected Python interpreter is the same one where Selenium was installed.

C3 – Driver/browser mismatch

Problem: The session fails with a message that the driver supports a different browser version.

Solution:

First check the installed browser version and make sure the driver is compatible with it.

With recent Selenium versions, Selenium Manager normally manages the driver automatically. Updating Selenium can therefore help:

```
python -m pip install --upgrade selenium
```

If a driver was downloaded manually, replace the old driver with a compatible version. It is also useful to check whether an old driver is still present in the system PATH.

C4 – Browser binary not found

Problem: Selenium cannot locate the browser executable.

Solution:

First make sure that the required browser is actually installed.

If the browser is installed in a non-standard location, its executable path can be given to Selenium.

For example:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.binary_location = r"C:\Path\To\chrome.exe"

driver = webdriver.Chrome(options=options)
```

The path should be replaced with the actual location of the browser executable.

C5 – Permission or antivirus blocking

Problem: The driver is downloaded but cannot execute, or security software quarantines it.

Solution:

First check Windows Security or the installed antivirus software to see whether the driver has been blocked or quarantined.

The following steps can help:

1. Check the antivirus quarantine or protection history.
2. Make sure the driver is stored in a location where it is allowed to run.
3. Use drivers obtained from trusted sources.

4. If the computer is managed by a college or company, contact the administrator if the driver needs approval.

Security software should not be permanently disabled just to run Selenium.

C6 – Proxy or firewall failure

Problem: Automatic driver download fails on a campus or corporate network.

Solution:

A college or company network may use a proxy or firewall that blocks Selenium Manager or pip from accessing external websites.

First check whether normal internet access is working. If the network requires a proxy, use the proxy settings provided by the network administrator.

If automatic downloading is blocked, an approved driver can be installed manually.

The error message should also be checked because it can help identify whether the problem is caused by the proxy, firewall, DNS, or authentication.

C7 – SSL certificate error during pip install

Problem: pip cannot verify a certificate while connecting to the package index.

Solution:

First check that the computer's date and time are correct.

It is also useful to update pip:

```
python -m pip install --upgrade pip
```

If the computer is connected to a college or company network that uses its own SSL certificate, the correct certificate should be configured according to the network administrator's instructions.

Disabling SSL certificate verification should not be used as a permanent solution because it reduces security.

C8 – Browser opens and immediately closes

Problem: The browser window appears briefly and disappears before the result can be inspected.

Solution:

This can happen because the Python program finishes immediately. It can also happen if an exception stops the program.

For debugging, we can temporarily wait for user input:

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

```
try:
```

```
    driver.get("https://example.com")
    input("Press Enter to close the browser...")
finally:
    driver.quit()
```

The terminal should also be checked for error messages or a traceback.

Using a long `sleep()` is usually not the best solution because it only hides the problem instead of showing what caused it.

C9 – Element not found

Problem: The page loads, but `NoSuchElementException` occurs for a visible element.

Solution:

First check whether the locator is correct. The element may also be loading dynamically or may be inside an `iframe`.

The following points should be checked:

1. Verify the locator.
2. Make sure the element belongs to the current page.
3. Check whether the element is inside an `iframe`.
4. Wait for the element if the page loads it dynamically.

For a dynamic page, an explicit wait can be used:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

element = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.ID, "username"))
)
```

If the element is inside an `iframe`, Selenium must first switch to that `iframe`.

C10 – Headless works differently

Problem: The script passes in normal mode but fails in headless mode.

Solution:

Headless mode does not show the browser window, and some browser settings such as the default window size may be different. This can sometimes cause elements to behave differently.

One possible solution is to set the browser window size explicitly:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless")
options.add_argument("--window-size=1920,1080")

driver = webdriver.Chrome(options=options)
```

We should also check for timing problems, scrolling issues, browser permissions, and elements that depend on the screen size.

Using explicit waits and comparing screenshots or logs from normal and headless execution can help identify the actual cause. ““

Part D – Mini Automation Task

Task:

Write a Selenium script that completes all of the following on a public practice site approved by your instructor:

- Open the site and verify the page title.
- Locate and interact with at least three different element types.
- Use at least two locator strategies.
- Use an explicit wait for a meaningful condition.
- Capture a screenshot after successful interaction.
- Handle one expected failure using exception handling and a useful error message.
- Close the session safely even if an error occurs.
- Add comments explaining why each locator and wait strategy was selected.
- Do not automate login to a real personal account.

Website Used:

<https://the-internet.herokuapp.com/login>

Python / Selenium Script:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import NoSuchElementException

driver = webdriver.Chrome()

try:
    # 1. Open the public Selenium practice website

    driver.get("https://the-internet.herokuapp.com/login")

    # Verify that the expected page title is displayed.

    expected_title = "The Internet"
    actual_title = driver.title

    if actual_title == expected_title:
        print("PASS: Page title verified:", actual_title)
    else:
```

```

    print("FAIL: Unexpected page title:", actual_title)

# Create an explicit wait.
# This waits for a meaningful browser condition instead of
# pausing for an arbitrary number of seconds.

wait = WebDriverWait(driver, 10)

# 2. Locate and interact with a TEXT INPUT
# ID is used because "username" is a stable and unique
# identifier for this input field.

username = wait.until(
    EC.visibility_of_element_located((By.ID, "username"))
)

username.send_keys("tomsmith")

print("PASS: Username field filled.")

# 3. Locate and interact with another TEXT INPUT
# ID is preferred because it uniquely identifies
# the password input.

password = driver.find_element(By.ID, "password")

password.send_keys("SuperSecretPassword!")

print("PASS: Password field filled.")

# 4. Locate and interact with a BUTTON
# CSS Selector is used here to demonstrate a second
# locator strategy.

login_button = wait.until(
    EC.element_to_be_clickable(
        (By.CSS_SELECTOR, "button.radius")
    )
)

login_button.click()

print("PASS: Login button clicked.")

# 5. Verify successful interaction

success_message = wait.until(
    EC.visibility_of_element_located(
        (By.CSS_SELECTOR, "div.flash.success")
    )
)

```

```

print("PASS: Successful interaction confirmed.")
print("Message:", success_message.text)

# 6. Capture screenshot

driver.save_screenshot("part_d_success.png")

print("PASS: Screenshot saved as part_d_success.png")

# 7. Demonstrate expected failure handling

try:
    # This element intentionally does not exist.

    driver.find_element(
        By.ID,
        "element_that_does_not_exist"
    )

except NoSuchElementException:

    print(
        "EXPECTED FAILURE: Test element was not found. "
        "The exception was handled correctly."
    )

finally:

    # Always close the entire browser session,
    # even if an unexpected error occurs.

    driver.quit()

    print("Browser session closed safely.")

```

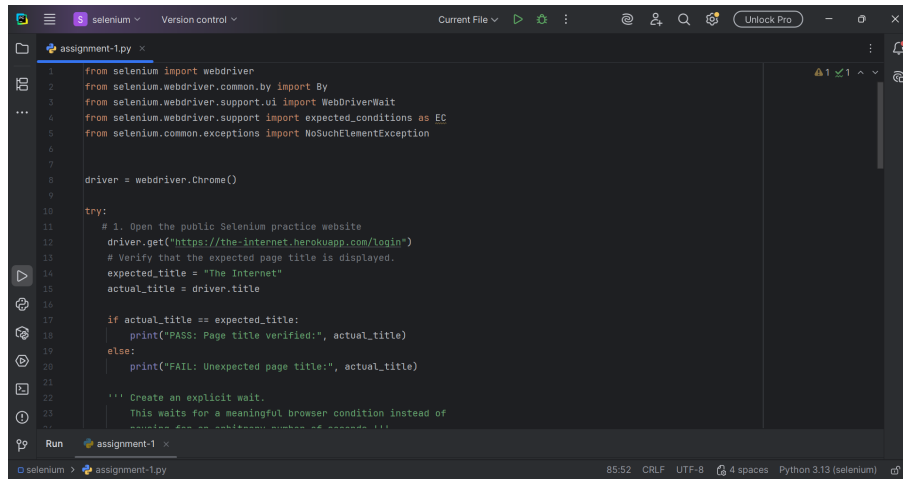
Expected Output:

```

PASS: Page title verified: The Internet
PASS: Username field filled.
PASS: Password field filled.
PASS: Login button clicked.
PASS: Successful interaction confirmed.
Message: You logged into a secure area!
PASS: Screenshot saved as part_d_success.png
EXPECTED FAILURE: Test element was not found.
The exception was handled correctly.
Browser session closed safely.

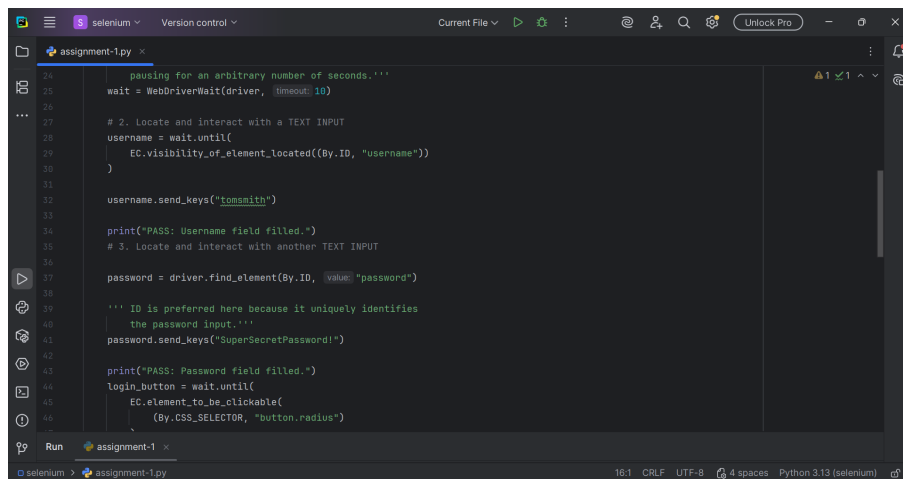
```

Screenshot / Evidence:



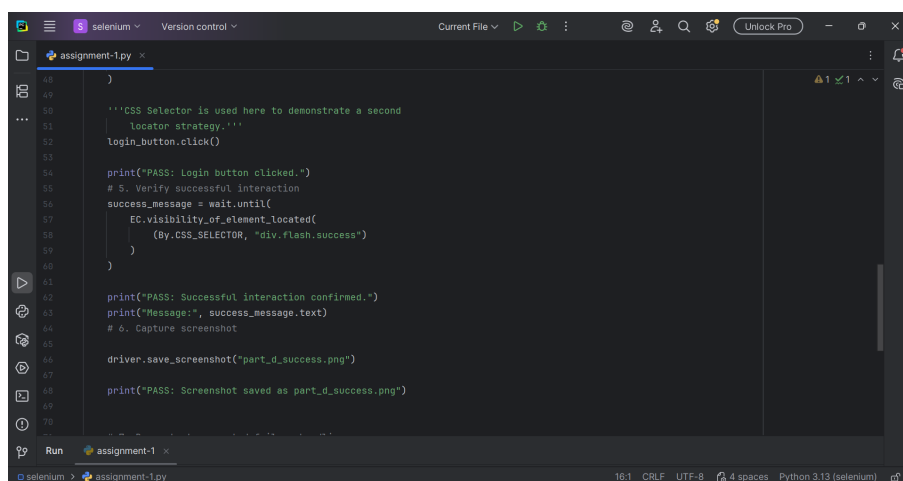
```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 from selenium.common.exceptions import NoSuchElementException
6
7
8 driver = webdriver.Chrome()
9
10 try:
11     # 1. Open the public Selenium practice website
12     driver.get("https://the-internet.herokuapp.com/login")
13     # Verify that the expected page title is displayed.
14     expected_title = "The Internet"
15     actual_title = driver.title
16
17     if actual_title == expected_title:
18         print("PASS: Page title verified:", actual_title)
19     else:
20         print("FAIL: Unexpected page title:", actual_title)
21
22     ''' Create an explicit wait.
23         This waits for a meaningful browser condition instead of
24         ... for an arbitrary number of seconds ...
```

Figure 4: Python script-1



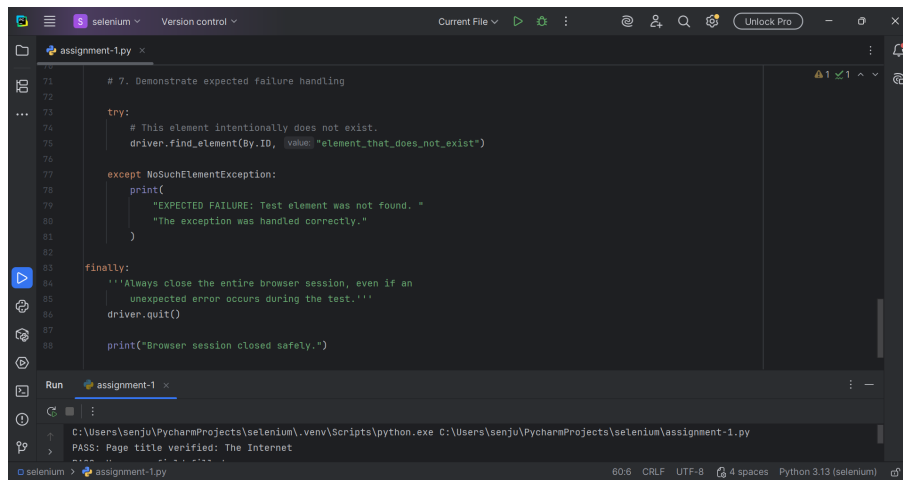
```
25     pausing for an arbitrary number of seconds.'''
26     wait = WebDriverWait(driver, timeout=10)
27
28     # 2. Locate and interact with a TEXT INPUT
29     username = wait.until(
30         EC.visibility_of_element_located((By.ID, "username"))
31     )
32
33     username.send_keys("tomsmith")
34
35     print("PASS: Username field filled.")
36
37     # 3. Locate and interact with another TEXT INPUT
38
39     password = driver.find_element(By.ID, value="password")
40
41     ''' ID is preferred here because it uniquely identifies
42         the password input.'''
43     password.send_keys("SuperSecretPassword!")
44
45     print("PASS: Password field filled.")
46     login_button = wait.until(
47         EC.element_to_be_clickable(
48             (By.CSS_SELECTOR, "button.radius")
49         )
50     )
```

Figure 5: Python script-2



```
51
52     '''CSS Selector is used here to demonstrate a second
53         locator strategy.'''
54     login_button.click()
55
56     print("PASS: Login button clicked.")
57
58     # 5. Verify successful interaction
59     success_message = wait.until(
60         EC.visibility_of_element_located(
61             (By.CSS_SELECTOR, "div.flash.success")
62         )
63     )
64
65     print("PASS: Successful interaction confirmed.")
66     print("Message:", success_message.text)
67
68     # 6. Capture screenshot
69
70     driver.save_screenshot("part_d_success.png")
71
72     print("PASS: Screenshot saved as part_d_success.png")
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

Figure 6: Python script-3



```
71 # 7. Demonstrate expected failure handling
72
73 try:
74     # This element intentionally does not exist.
75     driver.find_element(By.ID, value="element_that_does_not_exist")
76
77 except NoSuchElementException:
78     print(
79         "EXPECTED FAILURE: Test element was not found. "
80         "The exception was handled correctly."
81     )
82
83 finally:
84     '''Always close the entire browser session, even if an
85     unexpected error occurs during the test.'''
86     driver.quit()
87
88 print("Browser session closed safely.")
```

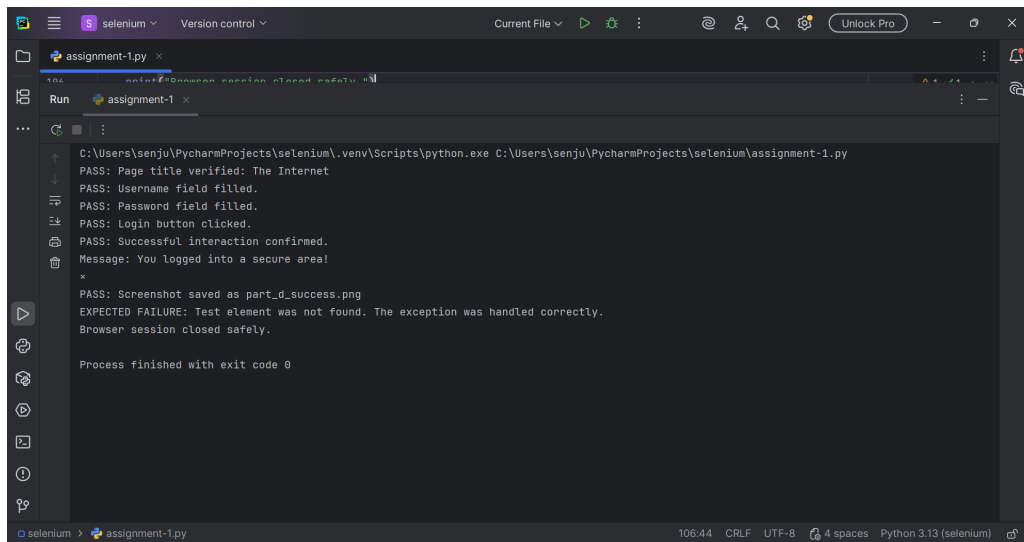
Run assignment-1

C:\Users\senju\PycharmProjects\selenium\.venv\Scripts\python.exe C:\Users\senju\PycharmProjects\selenium\assignment-1.py

PASS: Page title verified: The Internet

60.6 CRLF UTF-8 4 spaces Python 3.13 (selenium)

Figure 7: Python script-4



```
104 print("Browser session closed safely.")
105
C:\Users\senju\PycharmProjects\selenium\.venv\Scripts\python.exe C:\Users\senju\PycharmProjects\selenium\assignment-1.py
PASS: Page title verified: The Internet
PASS: Username field filled.
PASS: Password field filled.
PASS: Login button clicked.
PASS: Successful interaction confirmed.
Message: You logged into a secure area!
*
PASS: Screenshot saved as part_d_success.png
EXPECTED FAILURE: Test element was not found. The exception was handled correctly.
Browser session closed safely.

Process finished with exit code 0
```

108.44 CRLF UTF-8 4 spaces Python 3.13 (selenium)

Figure 8: execution output

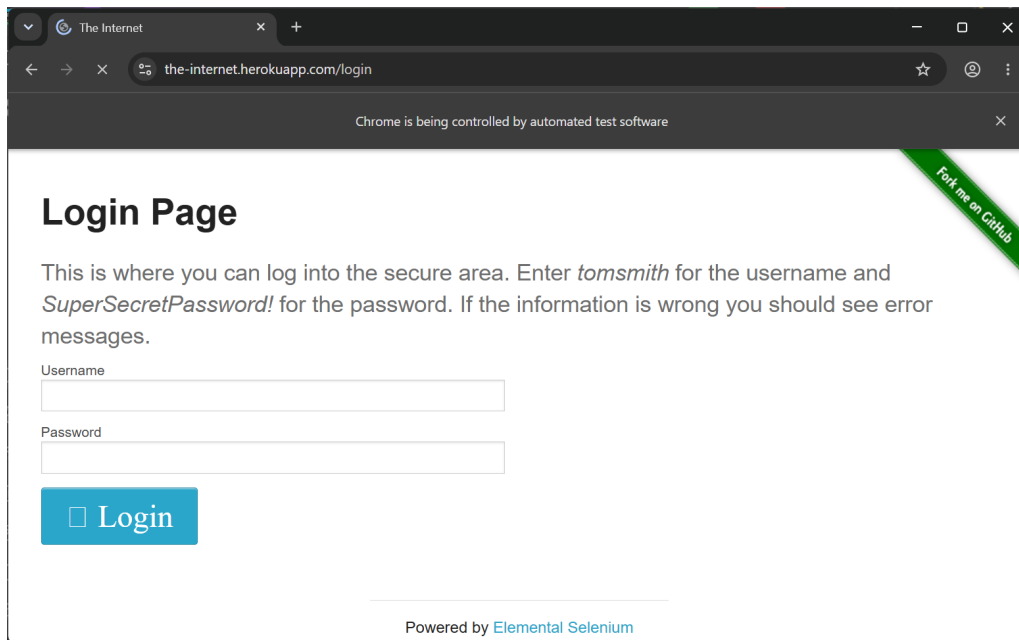


Figure 9: execution evidence

Reflection

R1. Which installation or execution issue required the most reasoning, and how did you isolate its cause?

Answer:The most challenging issue was the pip not being recognized / Selenium installation issue. I isolated the cause by checking Python and pip separately using `python --version` and `python -m pip --version`. I then used `python -m pip install selenium` to ensure that pip was connected to the correct Python installation. After installation, I verified it using `python -m pip show selenium` and tested Selenium with a simple Chrome WebDriver program.

R2. What would you change in your setup to make future Selenium projects easier to reproduce?

Answer:I would create a separate virtual environment for each Selenium project and install the required packages inside it. I would also maintain a `requirements.txt` file containing the package versions. This would make it easier to recreate the same environment on another computer or for another project.

R3. Which source was most useful, and how did you verify that its advice applied to your Selenium version?

Answer:The official Selenium documentation was the most useful source because it provides current information about WebDriver, installation, and Selenium APIs. I verified that the advice applied to my setup by checking my installed Selenium version with `python -m pip show selenium` and testing the recommended WebDriver code with my installed version
